

RMDB 数据库内核决赛答辩

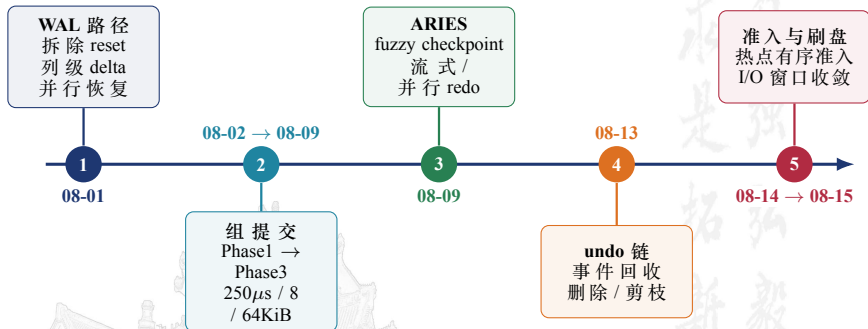
2026 年全国大学生计算机系统能力大赛 数据库管理系统设计赛

2026 年 8 月 18 日
珞珈山猫猫虫



武汉大学
WUHAN UNIVERSITY

五条主线：从 WAL 到并发收敛



日志体积 → 提交同步 → 恢复语义 → undo 内存 → 热点并发与 I/O

主线 1 + 2：日志路径与提交路径协同收敛

WAL / 恢复

- 整体移除 quiet-point WAL reset
- 消除 drain 事务、全量 flush 与 truncate 的周期停顿
- 表名哈希 + UPDATE 列级 delta，压缩 WAL
- 按表分桶并行 redo，并做负载均衡

阈值组提交

- 窗口内释放 latch，waiter 才能真正入批
- 放弃固定窗口与“等待流量空隙”两种过度等待
- 最终使用 $250\mu\text{s}$ / 8 waiter / 64 KiB 任一阈值触发
- 小批自举，waiter 立即登记，并与代数隔离

性能画像：fdatasync 占系统调用时间 86.5%

Syscall	Calls	Errors	Seconds	% time
fdatasync *	21,738	0	44.640517	86.5%
pread64	46,205	0	4.519137	8.8%
pwrite64	25,808	0	2.398639	4.6%
fsync *	34	0	0.023944	0.0%
openat	112	28	0.010988	0.0%
close	90	0	0.003732	0.0%
write	14	0	0.000543	0.0%

定位结论

21,738 次 fdatsync 累计耗时 44.64 s，远高于其他系统调用；组提交必须减少同步次数，并缩短其占用提交路径的时间。

主线 3 + 4: 恢复语义与 undo 链内存治理

ARIES 与流式恢复

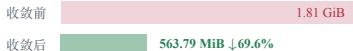
- CLR / ATT / DPT + repeating history, 支持恢复期间再次崩溃
- logical DELETE 精确跟踪; fuzzy checkpoint 与 16 MiB 分段 WAL
- 流式 redo、record view 零拷贝扫描与随机读缓存
- 恢复完成后并行重建索引, 并安全回收旧 WAL 段

undo 回收 / 删除 / 剪枝

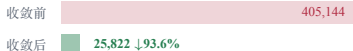
- refcount 1 $\rightarrow$ 0 事件进入 64 分片 ready queue
- 原子转移 undo ownership
- tombstone 原子条件删除, 不再等待全局 idle
- 链尾剪枝, 并由 walker 计数保护并发访问

I/O 收敛与最终长测：完整采样与原始长测图

I/O 写入量

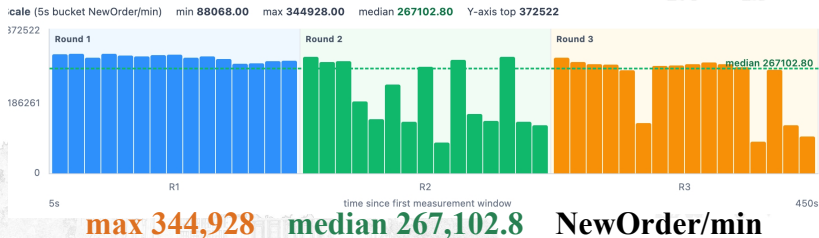


write syscall



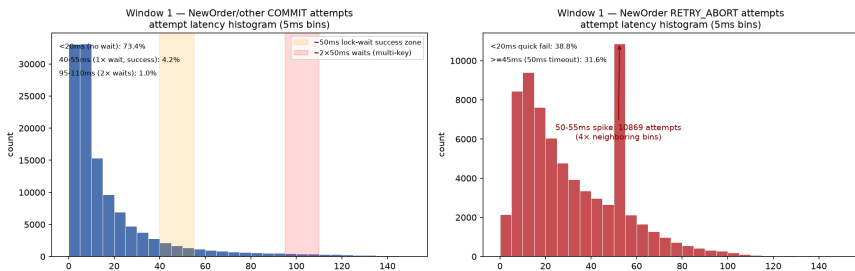
I/O 路径调整

- 512 页/tick 周期 writeback hint
- 禁用后台批量刷盘
- I/O 窗口优先交给提交路径
fdatasync



准入证据：50ms 等待在失败样本形成尖峰

50ms lock-wait fingerprint — sun/final_gc + chen-v5 Tester, local run 1 (W1)

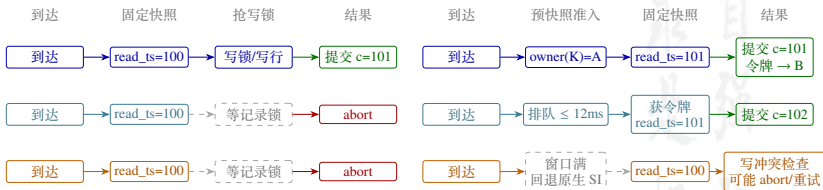


- 成功样本主要集中在 20ms 内；
- 失败样本在 50–55ms 出现 10,869 次尖峰；
- 固定等待直接塑造 abort 延迟指纹。

主线 5：准入——先排队，再取快照

改进前：先固定快照，再抢行锁

改进后：先准入排队，再固定快照



- 同快照并发写：1 提交、2 abort；有序准入后 A → B 顺序提交，C 回退原生 SI。
- 12ms / 3 waiter 维持有界窗口；超时或深队列回退原生 SI。
- response gate 保证同键客户端响应顺序。

- [3] K. Ren, J. M. Faleiro, and D. J. Abadi, "Design Principles for Scaling Multi-core OLTP Under High Contention," Proc. ACM SIGMOD, 2016.
- [4] G. Prasaad, A. Cheung, and D. Suciu, "Improving High Contention OLTP Performance via Transaction Scheduling," CoRR, 2018.
- [5] B. Ding, L. Kot, and J. Gehrke, "Improving Optimistic Concurrency Control Through Transaction Batching and Operation Reordering," Proc. VLDB Endow., 2018.
- [7] K. Ren, A. Thomson, and D. J. Abadi, "Lightweight Locking for Main Memory Database Systems," Proc. VLDB Endow., 2012.
- [8] D. Wang et al. (including W. Lu), "TXSQL: Lock Optimizations Towards High Contented Workloads," Companion of ACM SIGMOD/PODS, 2025.

机制 1：有界准入把快照放到正确的位置

提交前 | 原生 SI：先取快照，后等待记录锁



提交后 | 快照前有序准入：先排队，再创建新快照



- [2] H. Berenson, P. A. Bernstein, J. Gray, J. Melton, E. J. O'Neil, and P. E. O'Neil, "A Critique of ANSI SQL Isolation Levels," Proc. ACM SIGMOD, 1995.
- [8] D. Wang et al. (including W. Lu), "TXSQL: Lock Optimizations Towards High Contented Workloads," Companion of ACM SIGMOD/PODS, 2025.

从 TPC-C 热点模型到通用 SQL 结构识别

TPC-C 中的高频写热点

NewOrder	District 的 D_NEXT_O_ID 自增; 多个 Stock 行更新
Payment	Warehouse / District 的 YTD 累加; Customer 余额与计数更新
形态	UPDATE R SET c=c+? WHERE unique_key=?

内核启用准入的充分条件

- ① UPDATE 的每个 SET 都是同列 ADD/SUB;
- ② 参数化等值谓词能够形成逻辑键;
- ③ 完整唯一索引定位单行;
- ④ 首批请求发生在固定 SI 快照之前。

原始 SI 的浪费

多个事务先取得同一旧快照: 前驱提交后, 后继在真正写入阶段才发现 `latest_version > read_ts`, 随后 abort 并完整重试。

逻辑键身份

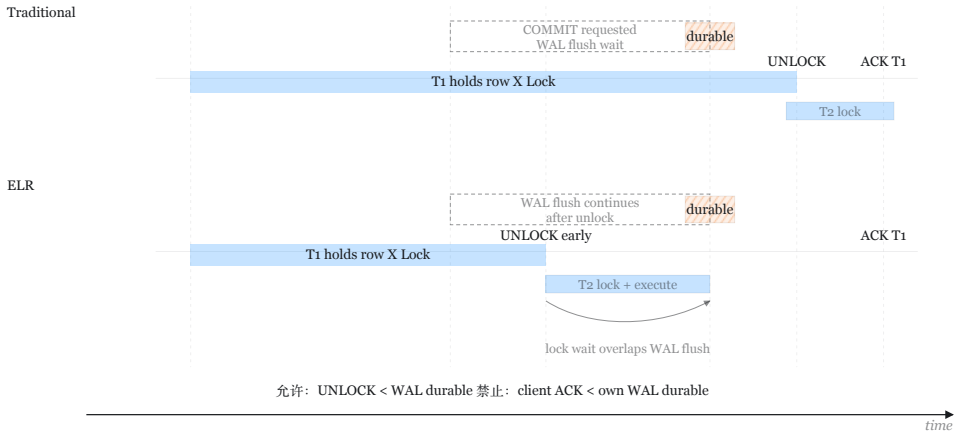
`relation + sorted(predicate columns) + typed values`

不使用事务名、业务表名白名单或参数序号;
NewOrder 与 Payment 对同一物理 District 能进入同一队列。

[1] Transaction Processing Performance Council, "TPC Benchmark C: Standard Specification, Revision 5.11," TPC, 2010.

[2] H. Berenson, P. A. Bernstein, J. Gray, J. Melton, E. J. O'Neil, and P. E. O'Neil, "A Critique of ANSI SQL Isolation Levels," Proc. ACM SIGMOD, 1995.

机制 2: Early Lock Release 将刷盘等待移出锁临界区



机制 3：内部提交顺序必须延伸到客户端观察顺序

(a) 单事务：快照、版本与 WAL 的三条并行轨迹

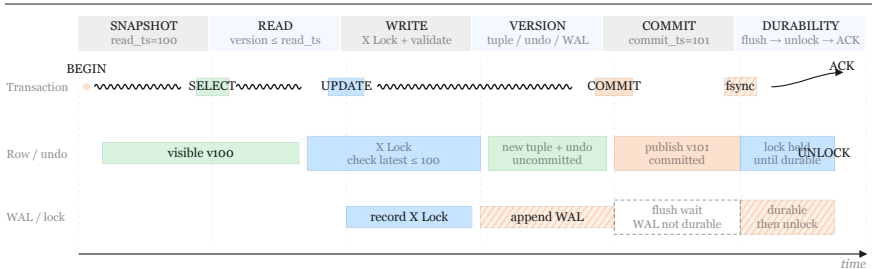


图 (a) 给出的单事务边界

COMMIT WAL 必须先 durable，客户端才能收到成功 ACK；ELR 只能提前 UNLOCK，不能提前 ACK。

多事务观察顺序的扩展

按实际写入序登记前驱 gate：同一会话下一请求作为隐式 ACK，阶段尾部使用 50ms 有界兜底。

- [2] H. Berenson, P. A. Bernstein, J. Gray, J. Melton, E. J. O'Neil, and P. E. O'Neil, "A Critique of ANSI SQL Isolation Levels," Proc. ACM SIGMOD, 1995.
- [6] R. Johnson, I. Pandis, R. Stoica, M. Athanassoulis, and A. Ailamaki, "Aether: A Scalable Approach to Logging," Proc. VLDB Endow., 2010.

热点写事务的三层协同机制

1. 冲突边界

快照前有序准入

可静态证明的热点写先排队，再取得 `read_ts`；减少“出生即陈旧”的事务。

2. 持久化边界

ELR + WAL fencing

COMMIT WAL 入队后先发布、先放锁；成功 ACK 仍等待依赖 WAL durable。

3. 观察边界

客户端响应 gate

把内部提交顺序延伸到客户端真正处理终态 evidence 的顺序。

三层机制作用在不同边界

- 冲突调度：作用于快照建立之前；
- **Early Lock Release**：作用于提交日志入序之后；
- **response gate**：作用于客户端观察成功之前。

[2] H. Berenson, P. A. Bernstein, J. Gray, J. Melton, E. J. O'Neil, and P. E. O'Neil, "A Critique of ANSI SQL Isolation Levels," Proc. ACM SIGMOD, 1995.

[6] R. Johnson, I. Pandis, R. Stoica, M. Athanassoulis, and A. Ailamaki, "Aether: A Scalable Approach to Logging," Proc. VLDB Endow., 2010.

[8] D. Wang et al. (including W. Lu), "TXSQL: Lock Optimizations Towards High Contented Workloads," Companion of ACM SIGMOD/PODS, 2025.

ACID 与收益

没有改变的语义

- 固定 SI snapshot
- 记录锁与写写冲突复查
- WAL-before-page
- 成功 ACK 必须 durable
- abort 后重跑完整事务

PostgreSQL: retry the complete transaction

得到的收益

- 大幅减少 stale-snapshot 重试风暴
- 后继获得包含前驱的新快照
- fdatasync 脱离热点锁区间
- 相邻提交共享 group flush
- 客户端证据顺序稳定

- [2] H. Berenson, P. A. Bernstein, J. Gray, J. Melton, E. J. O'Neil, and P. E. O'Neil, "A Critique of ANSI SQL Isolation Levels," Proc. ACM SIGMOD, 1995.
- [6] R. Johnson, I. Pandis, R. Stoica, M. Athanassoulis, and A. Ailamaki, "Aether: A Scalable Approach to Logging," Proc. VLDB Endow., 2010.

参考文献

资料	借鉴的思想	实际落地内容
TPC-C v5.11	校验事务语义和热点更新结构	只做通用 AST / 唯一点结构识别；不匹配事务名和业务表名
Berenson et al.	固定快照与 first-committer-wins 边界	保留 SI 写冲突；只把热点调度移到取得快照之前
ORTHRUS	执行前获知访问集合	不引入独立锁线程，仅从首批 prepared DML 提取逻辑写键
Strife	只调度高冲突集合，保留其他并行度	不做事务聚类；使用 12ms / 3 waiter 的在线有界窗口
OCC batching	先处理即将过期的写、减少无效读与 abort	不做 validator batching；只重排同键事务取得 snapshot 的时机
Aether / ELR	WAL 入序后早放锁，依赖者不得提前 ACK	增加 MVCC published-LSN 前沿与只读 snapshot barrier
VLL / TXSQL	轻量锁元数据、queue locking 与热点感知有序化	使用通用逻辑键的有界准入；不改造成 2PL group locking

[1] Transaction Processing Performance Council. *TPC Benchmark C: Standard Specification, Revision 5.11*. TPC, February 2010.

[2] H. Berenson, P. A. Bernstein, J. Gray, J. Melton, E. J. O’Neil, and P. E. O’Neil. “A Critique of ANSI SQL Isolation Levels.” *Proc. ACM SIGMOD*, pp. 1–10, 1995. doi:10.1145/223784.223785.

[3] K. Ren, J. M. Faleiro, and D. J. Abadi. “Design Principles for Scaling Multi-core OLTP Under High Contention.” *Proc. ACM SIGMOD*, pp. 1583–1598, 2016. doi:10.1145/2882903.2882958.

[7] K. Ren, A. Thomson, and D. J. Abadi. “Lightweight Locking for Main Memory Database Systems.” *Proc. VLDB Endow.*, 6(2):145–156, 2012. doi:10.14778/2535568.2448947.

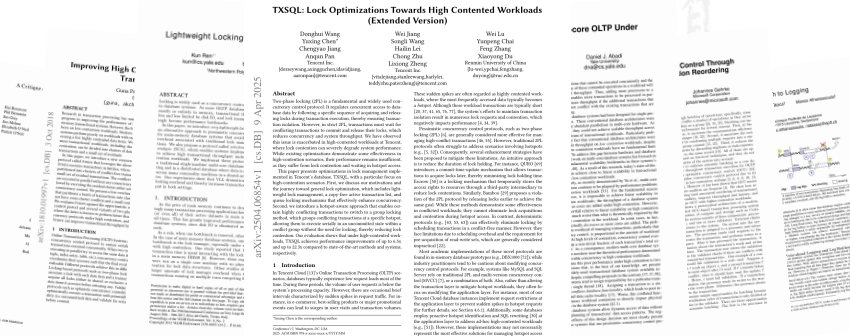
[4] G. Prasaad, A. Cheung, and D. Suciu. “Improving High Contention OLTP Performance via Transaction Scheduling.” *CoRR*, abs/1810.01997, 2018. arXiv:1810.01997.

[5] B. Ding, L. Kot, and J. Gehrke. “Improving Optimistic Concurrency Control Through Transaction Batching and Operation Reordering.” *Proc. VLDB Endow.*, 12(2):169–182, 2018. doi:10.14778/3282495.3282502.

[6] R. Johnson, I. Pandis, R. Stoica, M. Athanassoulis, and A. Ailamaki. “Aether: A Scalable Approach to Logging.” *Proc. VLDB Endow.*, 3(1):681–692, 2010. doi:10.14778/1920841.1920928.

[8] D. Wang et al. (including W. Lu). “TXSQL: Lock Optimizations Towards High Contented Workloads.” *Companion of ACM SIGMOD/PODS*, pp. 675–688, 2025. doi:10.1145/3722212.3724457.

参考文献



Snapshot Isolation Strife VLL TXSQL ORTHRUS OCC Batching Aether

优化总结

